Autonomous Maze Navigation with an iRobot Create 3 Using Infrared Sensing and LiDAR

Marjan Rahimof, Daniel Chiemezie Nwadike, Mark Rijpstra and Yehor Oblyvantsov

***Abstract*—Autonomous navigation through an unknown maze requires a robot to perceive walls, decide on a direction and recover from errors without human input. This paper presents [our solution] for an iRobot Create 3 running ROS 2 Jazzy, using its built-in infrared (IR) proximity sensors, bumpers and a 2D LiDAR. We compare [design A] with [design B] in [N] test runs in the same maze. [Key result with a number, e.g. success rate and mean completion time.] [One-sentence conclusion.]**

***Index Terms*—mobile robotics, maze navigation, wall following, finite-state machine, ROS 2, LiDAR**

I. Introduction

Autonomous navigation is a core capability of mobile robots: the robot must sense its surroundings, decide where to go and act on that decision in a closed loop [1], [2]. Maze solving is a classic, well-defined version of this problem. [Add 2-3 sentences on why mazes are a useful test case and which known strategies exist, with citations.]

A. Problem statement

The task is to program an iRobot Create 3 so that it drives autonomously from a start point to a finish point through a maze it has not seen before. The maze has a flat floor and walls at approximately 90° to each other, and it may contain loops. The start and finish are revealed only minutes before the demonstration.

This leads to the following research question: *[Which of the two controllers, A or B, reaches the finish more often and in less time in a previously unseen maze with loops?]*

B. System description

The platform consists of the components listed in Table I. The Create 3 publishes its sensor data and receives velocity commands over ROS 2 [3], [4]. Our control software runs as a Python node on the Raspberry Pi.

The seven IR sensors return a dimensionless reflection intensity; a larger value means a closer obstacle. They are fused into three directions: *front* is the maximum of the four front sensors, *left* is the maximum of the two left sensors (side_left and left), and *right* is the single right sensor. The maximum is used because the closest wall in a direction determines the robot's reaction.

All authors are with the Master Smart Systems Engineering, Hanze University of Applied Sciences, Groningen, The Netherlands. Introduction Project (SEVM25IP), Group 6.

Table I: Hardware and software used.

Component	Model / version	Role
Mobile base	iRobot Create 3	wheels, 7 IR proximity sensors, bumpers, odometry
Computer	Raspberry Pi 4	runs our control node
LiDAR	Slamtec RPLiDAR A1M8	360° distance measurement [5]
Operating system	Ubuntu 24.04	
Middleware	ROS 2 Jazzy, Cyclone DDS	communication between nodes
Language	Python [version]	control program

II. Design

A. Design A: reactive priority controller

The first design is purely reactive: at 10 Hz it reads the latest IR values and applies four rules in priority order (Table II). It keeps no memory of earlier decisions.

Table II: Rules of the reactive controller (Design A).

Priority	Condition	Action
1	$front > 300$	stop, rotate towards the more open side
2	$left > 200$	drive forward, steer right
3	$right > 200$	drive forward, steer left
4	otherwise	drive straight

B. Design B: left-hand wall follower as a finite-state machine

The second design implements the left-hand rule [1]: by keeping a wall on its left, the robot eventually reaches an exit that is connected to the outer wall. Behaviour is organised as a finite-state machine with the states FIND_WALL, FOLLOW_WALL, TURN_RIGHT, TURN_LEFT and BACK_UP. Hysteresis ($front > 300$ to start turning, $front < 150$ to stop) prevents oscillation, and a proportional controller keeps a constant distance to the left wall:

$$\omega = -K_p \left(left - left_{target}\right) \tag{1}$$

where ω is the angular velocity (rad/s) and K_p the proportional gain.

C. Design C: LiDAR-based navigation

[Describe the LiDAR design if it is used in the demo.]

D. Comparison of designs

[Compare the designs on memory, robustness to sensor noise, handling of loops and dead ends, and implementation effort, with references. Explain which design was chosen for the demonstration and why.]

III. Methods

A. Calibration of the IR thresholds

[Describe how the thresholds were measured: distance to wall in cm, number of readings, mean and standard deviation.]

B. Validation experiments

Each design was tested on the following sequence: a single straight wall, an inside corner, an outside corner, and a complete test maze. For each run we recorded whether the robot reached the finish, the completion time (s) and the number of bumper contacts. [Describe the test maze: size in m, layout, number of loops.]

The source code is available at [GitHub link].

IV. Results

Table III: Results of the validation runs in the test maze.

Design	Runs	Successful	Mean time (s)	Mean bumps

A: reactive
B: wall follower

[Report the demonstration result: round 1 and round 2.]

V. Discussion and conclusion

Interpret the results in III

Acknowledgements

Table IV: Contribution of each group member.

Member	Contribution	Share (%)
Marjan Rahimof		
Daniel Chiemezie Nwadike		
Mark Rijpstra		
Yehor Oblyvantsov		

References

[1] M. Ben-Ari and F. Mondada, *Elements of robotics*. Cham: Springer, 2018.

[2] P. Corke, *Robotics, vision and control: Fundamental algorithms in python*, 3rd ed. Cham: Springer, 2023.

[3] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.

[4] iRobot Education, "iRobot Create 3 documentation." https://iroboteducation.github.io/create3_docs/, 2026.

[5] Slamtec, "RPLIDAR A1 low cost 360 degree laser range scanner." https://www.slamtec.com/en/Lidar/A1, 2026.